14

Simplifying State Management with Vuex and Sending GET HTTP Requests

This chapter is about sending HTTP requests and solving the most common problem in big web applications—the problem of syncing the state of a component with another component. In large and complex applications, you need a tool that centralizes your application's state and makes the data flow transparent and predictable.

In this chapter, we will cover the following topics:

- Understanding complex state management
- Sending HTTP requests
- Setting up state management using Vuex

Technical requirements

You will need Visual Studio Code to complete this chapter.

The finished repository of this chapter can be found at `https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter14`.

Understanding complex state management

You will encounter state management when developing big and complex Angular, React, or Vue web applications. So what does state management mean?

Application **state management** is when your app starts to grow from just being a view or a couple of views. You're probably going to start running into the problem where you want to share some of that application state between different and nested components. An example of that is when you have to create a mechanism where two deep components should always sync. See *Figure 14.1* for a diagram of this:

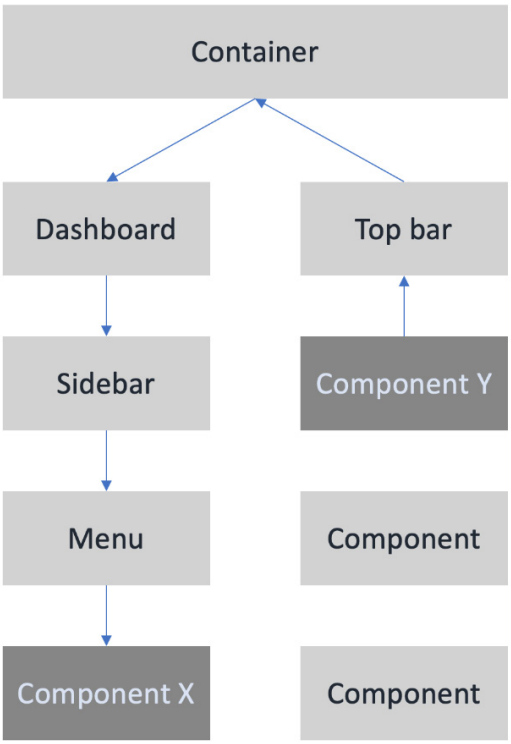

Figure 14.1 – Shared component's local state

Figure 14.1, shows that **Component Y** shares state with **Component X**. Now, there are different ways you could do that. One is bypassing events and passing in properties from a top-level component. Even though you can pass props and events, passing events and props in multiple nested components makes an application hard to maintain and makes the code hard to reason about.

Understanding global state

So what is the solution for unmaintainable code due to the sharing of states between multiple nested components? An application-wide state or global state, which is also known as the **store**. The idea in a global state solution is that every view and every component can just be a reflection of that one central state. See *Figure 14.2*:

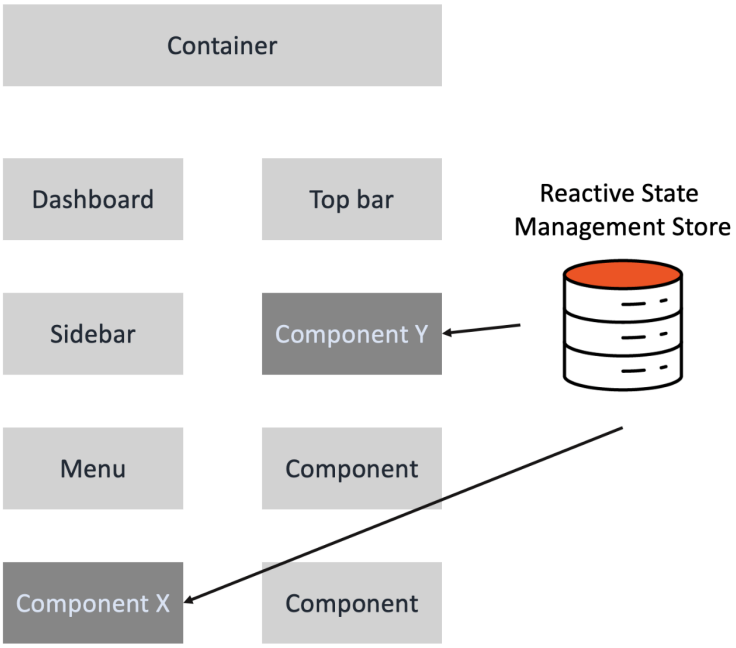

Figure 14.2 – Reactive global state

Figure 14.2 shows a reactive global state that is accessible to any components. Having a store is good. We only have one source of truth, and everything is just a reflection of that. The reactiveness of the global state or store is a lovely way of making sure that everything the user sees in our application always stays in sync.

So how do you set up a store in Vue.js? We will do that later, but let's do some simple data fetching without a store in the next section. The simple data fetching will prove that we can send an HTTP GET request to our ASP.NET Core Web API.

Sending an HTTP request in Vue.js

Sending HTTP requests to a RESTFul service if you are developing a modern web application is trivial. There are HTTP client libraries such as `api-sauce`, `super-agent`, and `axios`. Do you know that JavaScript itself has a native API for sending HTTP requests? Yes, and that is the Fetch API.

The Fetch API is available in JavaScript and TypeScript. However, it only works in modern browsers and will not work when you load your application in older browsers. Not only that, but the Fetch API can also be too verbose when using its APIs to send a request, in my opinion. I would prefer an HTTP client library with an abstraction to not write extra code such as `res.json()`, `headers`, or the `property` method. In that case, we will be using Axios as our HTTP client library.

Axios is a promise-based HTTP client library for the browser and server-side Node.js app. It is simple to use and also supports older browsers. We will install Axios together with Day.js, which is a library for manipulating dates and times.

So, go to your `vue-app` root directory and run the following `npm` command:

```
npm i axios dayjs
```

The preceding `npm` command will install `axios` and `dayjs`.

Next, we create a folder named `api` in the `src` directory.

Create a JavaScript file, `api-v1-config.js`, inside the `api` folder and write the following code:

```
import axios from "axios";
const debug = process.env.NODE_ENV !== "production";
const baseURL = debug
  ? "https://localhost:5001/api/v1.0/"
  : "https://traveltour.io/api/v1.0/";
let api = axios.create({ baseURL });
export default api;
```

The preceding code is an Axios setup for API version one of our ASP.NET Core Web API. We are creating an instance of Axios and passing a base URL for it to use.

Let's create another JavaScript file, `api-v2-config.js`, in the `api` folder and write the following code:

```
import axios from "axios";
const debug = process.env.NODE_ENV !== "production";
const baseURL = debug
? "https://localhost:5001/api/v2.0/"
: "https://traveltour.io/api/v2.0/";
let api = axios.create({ baseURL });
export default api;
```

The preceding code is an Axios configuration for API version two of our ASP.NET Core Web API. You will notice that the only thing that is different here is the version.

Create another JavaScript file, `weather-forecast-services.js`, inside the `api` folder and write the following code:

```
import api from "@/api/api-v2-config";
export async function getWeatherForecastV2Axios(city) {
  return await api.post(`WeatherForecast/?city=${city}`);
}
```

The preceding code is a service for sending a POST request to the `WeatherForecast v2` controller. We named the default export of the Axios configuration `api` and then used it to call a `POST` method to the `WeatherForecast` endpoint. I prefer adding an `axios` suffix to the function service to help me recognize the right file to import while reading the IntelliSense of my IDE or code editor.

Now let's update the `WeatherForecast.vue` component of the `views | AdminDashboard` folder. Update the contents of the folder with the following code:

```
<script>
import { getWeatherForecastV2Axios } from "@/api/weather-
  forecast-services";
export default {
  name: "WeatherForecast",
  async mounted() {
    await getWeatherForecastV2Axios("Oslo");
```

```
    },
};
</script>
```

In the preceding code, we import the `getWeatherForecastV2Axios` service and use it in `mounted()` life cycle hooks so that it gets triggered in advance before the DOM gets rendered.

Run the backend application and the frontend application by running the following command inside the `Travel.WebApi` project:

```
dotnet run
```

Wait for a few seconds after running the preceding `dotnet` command, then go to your browser, type `https://localhost:5001`, and check the Chrome DevTool, as shown in the following screenshot:

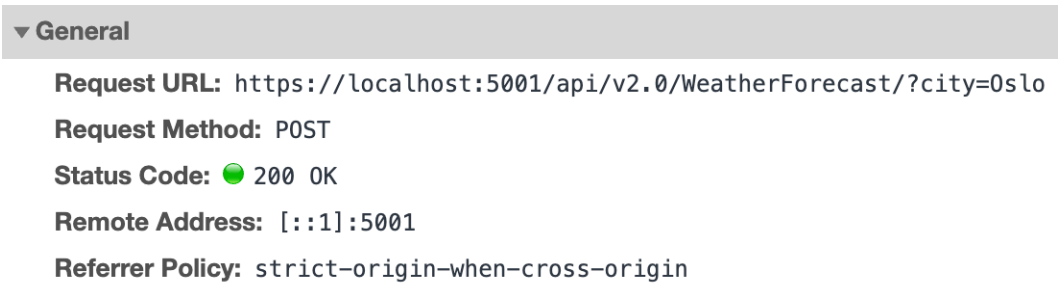

Figure 14.3 – Chrome DevTool status after sending the POST request to the WeatherForecast API

Figure 14.3 shows that the **POST** request returns a **200 OK** status code, which means we get what we ask for, and that would be the following JSON results:

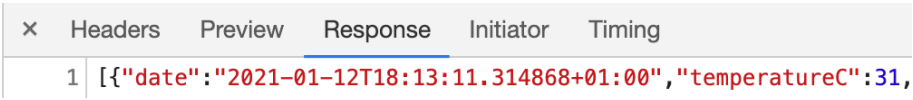

Figure 14.4 – JSON response after sending the POST request to the WeatherForecast API

The JSON response from the `WeatherForecast` controller can be seen in *Figure 14.4*. The response signifies that we can send a request and get a response from our backend.

Now we know that there's data in our browser that we can use in a user interface, let's build a UI for our data.

Let's update `WeatherForecast.vue` again.

Let's bring in `relativeTime` and `dayjs` from the `dayjs` library we installed:

```
import relativeTime from "dayjs/plugin/relativeTime";
import dayjs from "dayjs";
```

`relativeTime` formats the date to relative time strings such as `an hour ago` or `in 2 days`.

Now let's update the mounted life cycle hook:

```
  async mounted() {
    this.loading = true;
    dayjs.extend(relativeTime);
    await this.fetchWeatherForecast(this.selectedCity);
    this.loading = false;
  },
```

We are using the library `dayjs` here to manipulate the date. You will also notice that we are using the local state `loading` by setting it to true then setting it to false after fetching the data. We are going to use the local state `loading` to show a spinner component on the screen of our app.

Now let's define our local states:

```
  data() {
    return {
      weatherForecast: [],
      cities: [],
      selectedCity: "Oslo",
      loading: false,
    };
  },
```

We have a `weatherForecast` array, a `cities` array, a `selectedCity` string, and a `loading` Boolean in our local states. They are all initialized with default values.

Let's add an asynchronous method inside a `methods` object like so:

```
  methods: {
    async fetchWeatherForecast(city = "Oslo") {
      this.loading = true;
      try {
```

```
        const { data } = await getWeatherForecastV2Axios(
          city);
        console.log(data);
        this.weatherForecast = data?.map((w) => {
          const formattedData = { ...w };
          let date = w.date;
          formattedData.date = dayjs(date).fromNow();
          return formattedData;
        });
      } catch (e) {
        alert("Something happened. Please try again.");
      } finally {
        this.loading = false;
      }
    },
  },
```

We are setting `loading` to `true` for our spinner in this function, sending the request using the `getWeatherForecastV2Axios` service. After getting the data, we are going to use the JavaScript array utility `map`, which is like a loop to format each object's date from the JSON array response.

Let's also create another method for returning a color-coded temperature. Get the complete function in the GitHub repo because it is too big to write it here:

```
  getColor(summary) {
      switch (summary) {
        case "Freezing":
          return "indigo";
        // get the rest from the github
        default:
          return "grey";
      }
    },
```

The method is essentially just a helper that returns a color depending on the temperature.

Now for the `template` syntax. Here it is:

```
<template>
  <v-container>
    <div class="text-h4 mb-10">
      Two-week weather forecast of different cities
    </div>
    <div class="v-picker--full-width d-flex justify-center"
       v-if="loading">
      <v-progress-circular
        :size="70"
        :width="7"
        color="purple"
        indeterminate
      ></v-progress-circular>
    </div>
    <!-- Insert <v-simple-table> here -->
  </v-container>
</template>
```

The preceding code is the `title`, `container`, and the spinner UI of our application for the `WeatherForecast` page.

Now include the `v-simple-table` component in the container of the `WeatherForecast` UI. Insert the component in the place where you will find the `<!-- Insert <v-simple-table> here -->` comment:

```
    <v-simple-table>
      <template v-slot:default>
        <thead>
          <tr>
            <th class="text-left">Dates</th>
            <th class="text-left">City</th>
            <th class="text-left">&#8451;</th>
            <th class="text-left">&#8457;</th>
            <th class="text-left">Summary</th>
          </tr>
        </thead>
```

```
          <tbody>
            <tr v-for="item in weatherForecast"
              :key="item.date">
              <td>{{ item.date }}</td>
              <td>{{ item.city }}</td>
              <td>{{ item.temperatureC }}</td>
              <td>{{ item.temperatureF }}</td>
              <td>
                <v-chip :color="getColor(item.summary)" dark>{{
                  item.summary
                }}</v-chip>
              </td>
            </tr>
          </tbody>
        </template>
      </v-simple-table>
```

The `v-simple-table` component will be the table of `WeatherForecast`, which shows the temperature for a given date in Oslo:

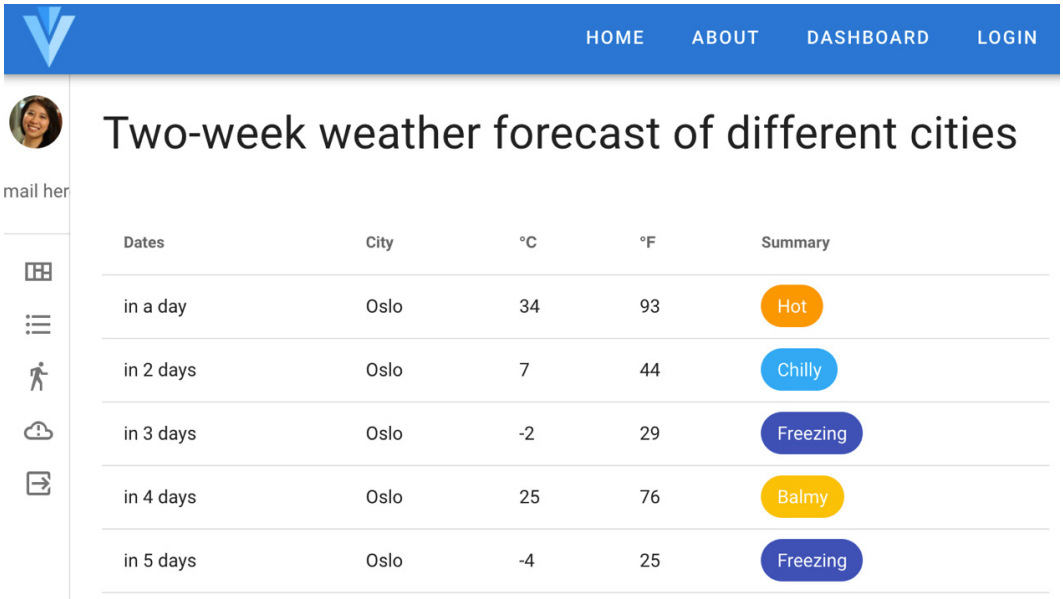

Figure 14.5 – Two-week weather forecast

The preceding forecast is a two-week forecast for **Oslo**. There you can see the dates, cities, temperatures in Celsius and Fahrenheit, and a visual summary of the weather.

In the next section, we are going to add a drop-down selector where we can select a city, and the cities we can choose will come from our ASP.NET Core Web API.

Now we have a `WeatherForecast` feature using a simple HTTP request without using a store. The feature that we are going to build in the next section will be using a Vuex state management library. We are going to reuse the functionality in another component that requires a store in the next chapter, *Chapter 15*, *Sending POST, DELETE, and PUT HTTP Requests in Vue.js with Vuex*.

Setting up state management using Vuex

Vuex is the official state management library for Vue.js that is widely used for managing complex components. Vuex has a reactive global store and is reasonably easy to set up. I will explain the parts of the Vuex implementation as we write the code.

But before we start building our store, let's remove the authorization from the API controller so that we don't need an auth token when sending requests to the `api/v1.o/` endpoints.

To do that, go to the `ApiController.cs` file in the namespace `Travel.WebApi.Controllers.v1` of the `Travel.WebApi` project and comment the `Authorize` attribute like so:

```
// [Authorize]
```

After commenting the `Authorize` attribute, we can now use `api/v1.0/` temporarily.

Let's start by setting up the update part of the Vuex.

Step 1 – Writing a store

Create a folder named `tour` inside the `store` folder. It will be like this: `src` | `store` | `tour`.

Step 2 – Writing a module

Create a JavaScript file named `services.js` inside the `tour` folder and write the following code:

```
import api from "@/api/api-v1-config";
export async function getTourListsAxios() {
```

```
  return await api.get("TourLists");
}
```

We are creating a service that sends a request to the version one `TourLists` controller or endpoint.

Step 3 – Writing a module if we are using TypeScript

Create a JavaScript file named `types.js`, still inside the `tour` folder, and then write the following code:

```
export const LOADING_TOUR = "LOADING_TOUR";
export const GET_TOUR_LISTS = "GET_TOUR_LISTS";
```

The preceding code is not a requirement in writing Vuex but became a part of the design when implementing state management. The preceding types are called **action** types, and we are going to the types to let the store know what kind of action it should take in the state after receiving an action. The **snake-casing** types are just strings, but they can prevent typo errors when writing actions, which you will see later.

Step 4 – Writing an API service

Create a JavaScript file named `actions.js` inside the `tour` folder and then write the following code:

```
import * as types from "./types";
import { getTourListsAxios } from "@/store/tour/services";
// asynchronous action using Axios
export async function getTourListsAction({ commit }) {
  commit(types.LOADING_TOUR, true);
  try {
    const { data } = await getTourListsAxios();
    commit(types.GET_TOUR_LISTS, data.lists);
  } catch (e) {
    alert(e);
    console.log(e);
  }
  commit(types.LOADING_TOUR, false);
}
```

The action carries instructions on how to update the global state of the store. Action is terminology that you can also find in the state management libraries of other JavaScript frameworks. Hence, it is valuable to remember this.

Moreover, actions can contain asynchronous operations if needed. We are defining an `action` function with a deconstructed `commit` parameter to commit to a mutation.

To use the commit, simply pass the action types as the first argument. If we need a payload in that particular commit, we can use the second optional argument to pass the payload. An example of that is `commit(types.LOADING_TOUR, true);`. In the previous sentence, the line of code is a `commit` statement about loading `tour` with a Boolean argument as `payload` of the action.

We are not limited to writing a single commit in an `action` function. We can write one or more commits as long as we need to.

What we are doing in `getTourListAction` of `actions.js` is enabling `loading tour`. The goal is to create a spinner while we are fetching the data using the `getTourListsAxios` service. We will then use the response in `commit(types.GET_TOUR_LISTS, data.lists)` to keep the data in our global state essentially.

We then passed `false` to `loading tour` to stop the spinner from spinning.

You will see more about this later when we write more actions in the upcoming chapters.

Step 5 – Writing an action type

Create a JavaScript file named `state.js` inside the `tour` folder and then write the following code:

```
const state = {
  lists: [],
  loading: false,
};
export default state;
```

The `state` object will be part of our app's global state, which is the store. A store is a big object with properties that have default or initial values. The initial values or initial states are something that you have to remember because they are also applicable in any other state management libraries in different JavaScript frameworks.

Step 6 – Writing an action

Create another JavaScript file named `mutations.js` inside the `tour` folder and then write the following code:

```
import * as types from "./types";
const mutations = {
  [types.GET_TOUR_LISTS](state, lists) {
    state.lists = lists;
  },
  [types.LOADING_TOUR](state, value) {
    state.loading = value;
  },
};
export default mutations;
```

Mutations are functions that get triggers depending on the type of actions that get dispatched. They perform the actual state modifications. As you can see in the preceding code, `types.GET_TOUR_LISTS` changes the value of `state.lists`. The `state` parameter, which is the first parameter, is a state that is part of the global state. It is automatically passed here by Vuex.

The second parameter is the optional payload that comes from the action we defined earlier.

Also, the job of the mutation is to update a part of the global state directly. It depends on whether you want to delete or update an existing state.

Step 7 – Writing a state

Create a JavaScript file named `getters.js` inside the `tour` folder, and then write the following code:

```
const getters = {
  lists: (state) => state.lists,
  loading: (state) => state.loading,
};
export default getters;
```

Getters are the computed properties or derived values for stores. The result or output of `getters` is cached based on its dependencies and will rerun or re-evaluate when any of its dependencies have changed. `getters` are what we will include in our component to connect the global states to our UI.

Step 8 – Writing a mutation

Create a JavaScript file named `index.js` in the `tour` folder. This file will be in the index file of our `tour` module. Once the file is created, we write the following code:

```
import state from "./state";
import getters from "./getters";
import mutations from "./mutations";
import * as actions from "./actions";
export default {
  namespaced: true,
  getters,
  mutations,
  actions,
  state,
};
```

We import the `state` file, `getters` file, `mutations` file, and the `actions` file for the `index` file of our `tour` module or namespace. After importing the necessary files, we are exporting them and including a `namespaced` property set to `true`.

Modules or **namespaces** are divisions inside the store. Each module or namespace has its `actions`, `state`, `mutations`, `getters`, and even nested modules.

Step 9 – Writing a getter

Update the `index.js` file of the store with the following code. The code is where we are going to add the `tour` module:

```
import Vue from "vue";
import Vuex from "vuex";
import createLogger from "vuex/dist/logger";
import tourModule from "./tour";
Vue.use(Vuex);
const debug = process.env.NODE_ENV !== "production";
```

```
const plugins = debug ? [createLogger({})] : [];
export default new Vuex.Store({
  modules: {
    tourModule,
  },
  plugins,
});
```

We import a logger, the tour module in the preceding file, and then remove the following properties: `state`, `mutations`, and `actions`. We then use `tourModule` as the property of the `modules` object.

The `modules` object is where all the modules of our Vue.js app are located. That means we will update the `modules` object as we add a new module or namespace in our Vue.js application.

Step 10 – Updating the store by inserting the module

Now let's update the `WeatherForecast.vue` page. Also, let's import `mapActions` and `mapGetters` from `vuex`:

```
import { mapActions, mapGetters } from "vuex";
```

`mapGetters` is a helper that simply maps local computed properties to store `getters`. Similarly, `mapActions` is a helper that maps store dispatch to the component methods. We will be using these mappers in the `methods` block and `computed` block.

Insert `mapActions` inside the first line of the `methods` block. Use the `spread` operator, the three dots prefixing `mapActions` like so:

```
methods: {
    ...mapActions("tourModule", ["getTourListsAction"]),
    …
  },
```

`mapActions` maps `this.getTourListsAction`, a `local` method that is automatically created, to `this.$store.dispatch("getTourListsAction")`. We can now use `this.getTourListsAction` to trigger an action for mutation.

Before using `getTourListsAction`, we would also want to access the `lists` state of our store. To do that, create `computed` block and insert `mapGetters` using a spread operator like so:

```
  computed: {
    ...mapGetters("tourModule", {
      lists: "lists",
    }),
  },
```

`mapGetters` maps `this.lists`, a local state that is automatically created, to `this.$store.getters.lists`.

Step 11 – Updating components with mapGetters and mapActions

Let's update the `mounted` method with the following code. We are going to add two new lines of code:

```
  async mounted() {
    …,
    await this.getTourListsAction();
    this.cities = this.lists.map((pl) => pl.city);
  },
```

We are triggering `getTourListsAction` and then extracting the cities from the lists and storing them in the cities' local state.

Now, for the dropdown or *select* UI of the `WeatherForecast` screen, add the `v-select` component on top of the `v-simple-table` component in the `template` syntax area:

```
    <v-select
      @change="fetchWeatherForecast"
      v-model="selectedCity"
      :items="cities"
      label="City"
      persistent-hint
      return-object
      single-line
```

```
        clearable
      ></v-select>
```

The preceding component will give the `WeatherForecast` screen a functionality where we can choose which city to check for a 14-day weather forecast.

Before running the ASP.NET Core and the Vue.js app, I want you to use the SQLite database of the `Travel.WebApi` project. I have updated the data inside it to help us see what we are working on in the frontend in terms of design. You will see the JSON equivalent of the SQLite data in the GitHub repo `https://github.com/PacktPublishing/ASP.NET-Core-5-and-Vue.js-3/tree/master/Chapter-14`, to let you know what date is currently in the database and the shape of it.

Delete the `TravelTourDatabase.sqlite3` database file you have right now in your project and replace it with the one from the GitHub repository.

Rerun the application by running the following command inside the `Travel.WebApi` project:

```
dotnet run
```

Wait for a couple of seconds, then check your browser. You should see the drop-down menu selector as shown in the following screenshot:

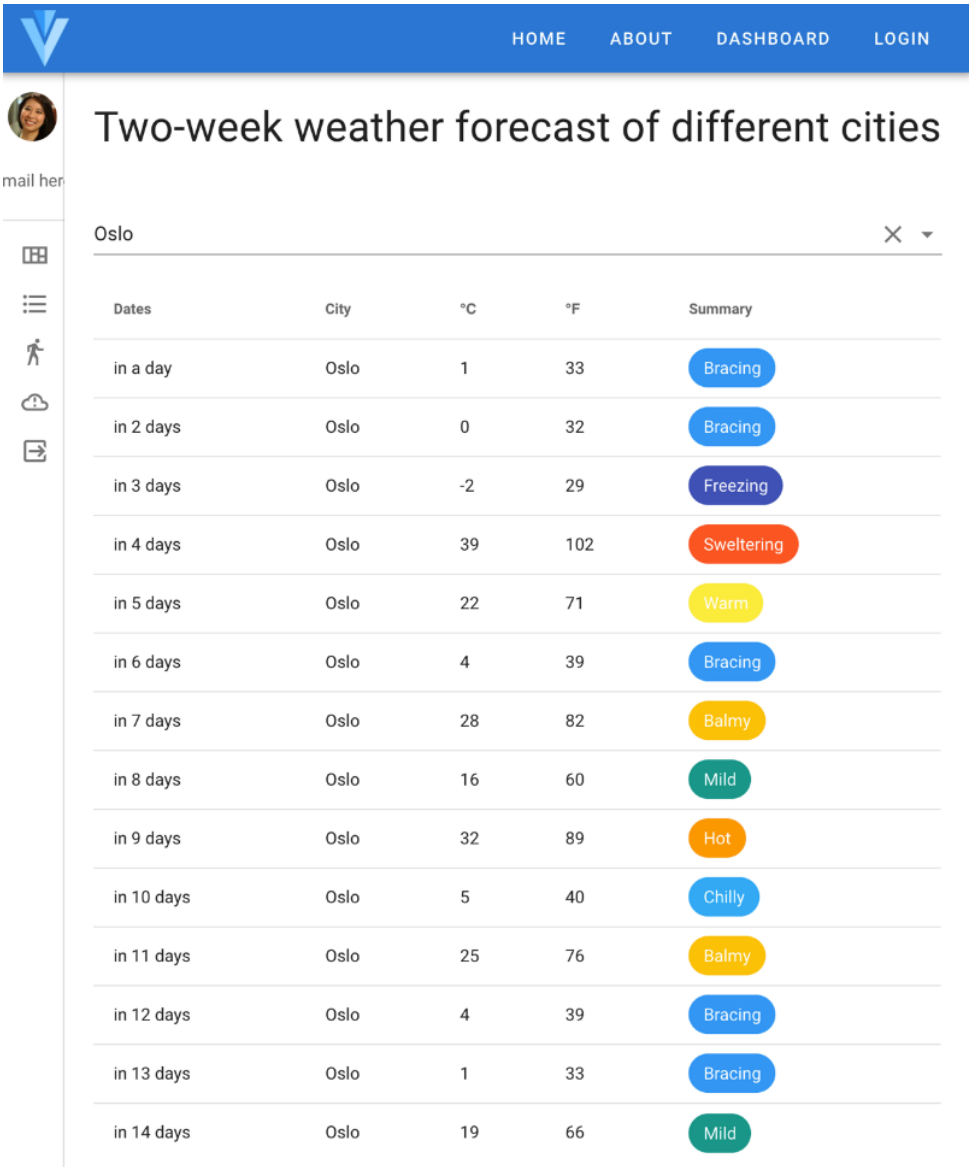

Figure 14.6 – Vuetify select component

Figure 14.6 shows the selector with the default **Oslo** as the selected city. Try clicking it to see the other available cities.

Next, check out the console logs on your browser's DevTools. You should also see the Vuex logger plugin that we included in the store setup. It will look something like what's shown in *Figure 14.7*:

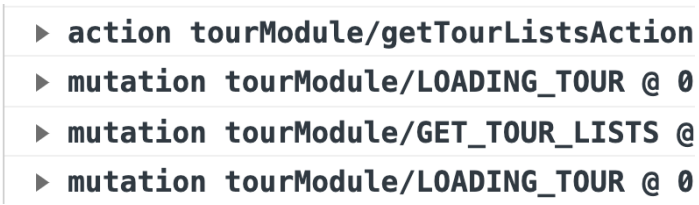

Figure 14.7 – Vuex logger

The Vuex logger in the preceding screenshot shows the logs of what's happening inside the Vuex state management of the Vue.js application. We can use this to debug our application if there are unexpected behaviors in our application while using the Vuex store's parts.

Using Vuex and other state management libraries in other frameworks takes a lot of code to set them up. But once you've finished setting up all the moving parts and your store's configuration, adding new actions and using them becomes easy because you just need to update the existing code. You are only going to write the setup once, and the rest is seamless.

Even though the setup takes time to finish, the benefits where you don't have to pass down props and events in and out of nested components are so good and will make your life easier when developing complex synchronization of states between nested components.

I suggest remembering the preceding flow of topics to have a checklist of what to do whenever you are implementing Vuex in your project. Now let's recap what you have learned in this chapter.

Summary

This chapter might be one of the most challenging chapters so far because of the state management concept. Nevertheless, learning how to do state management is invaluable. You have learned how to send HTTP requests using Axios. You have discovered the idea of state management in big applications.

You have also learned how to use Vuex and Vuex's parts, such as the store, modules, actions, mutations, and getters.

In the next chapter, we will build functionalities for sending *POST*, *DELETE,* and *PUT HTTP* requests in Vue.js with Vuex.